

目录

- 神经网络语言模型
 - 前馈神经网络模型
 - 循环神经网络模型
- Word2vec词向量
- Glove词向量
- 评价与应用

□ N-gram Language Model (bigram)

$$P(w_1 w_2 \cdots w_n) = \prod_{i=1}^n P(w_i | w_{i-1})$$

□ 缺点：

- 模型容易受到数据稀疏的影响，一般需要对模型进行平滑处理
- 无法对长度超过N的上下文依赖关系进行建模

神经网络语言模型 (Neural Network Language Model) 在一定程度上克服了这些问题

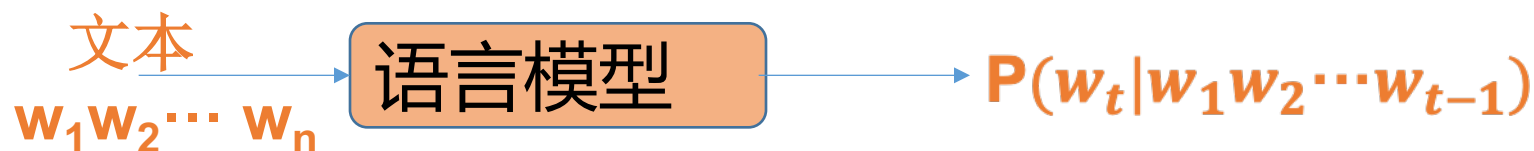
神经网络语言模型：

- ① 引入词的分布式表示(词向量) 缓解数据稀疏
- ② 引进更先进的神经网络模型结构（如循环神经网络、Transformer等），可以对长距离上下文依赖进行有效的建模

神经网络语言模型几乎已经替代N元语言模型

在大规模无标注文本数据上进行静态词向量的预训练

预训练任务



类别标签集合的**分类**问题

□**输入**: 历史词序列 $w_1 w_2 \cdots w_{t-1} (w_{1:t-1})$

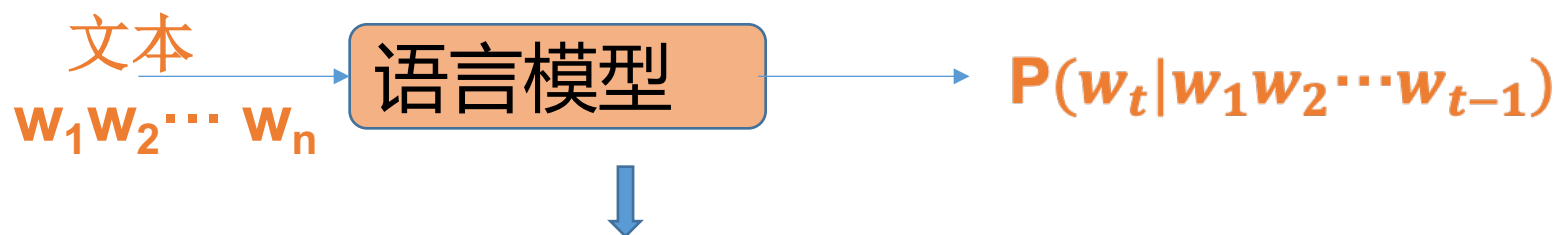
□**输出**: 目标词 w_t

□从**无标注**的文本语料中构建**训练数据集**，并通过优化该数据集上的**分类损失**(如**交叉熵损失**或**负对数似然损失**)对模型进行训练

□由于监督信号来自数据自身，因此这种学习方式也被称为**自监督学习** (Self-supervised Learning)

如何处理动态长度的历史词序列（模型输入）？

前馈神经网络语言模型



马尔可夫假设—对下一个词的预测只与历史中最近的 $n-1$ 个词相关

$$P(w_t | w_1 w_2 \cdots w_{t-1}) = P(w_t | w_{t-n+1:t-1})$$

模型的输入: 长度为 $n-1$ 的定长词序列 $w_{t-n+1:t-1}$

模型的任务: 对条件概率 $P(w_t | w_{t-n+1:t-1})$ 进行估计

前馈神经网络语言模型

输出层

w_t

输入层

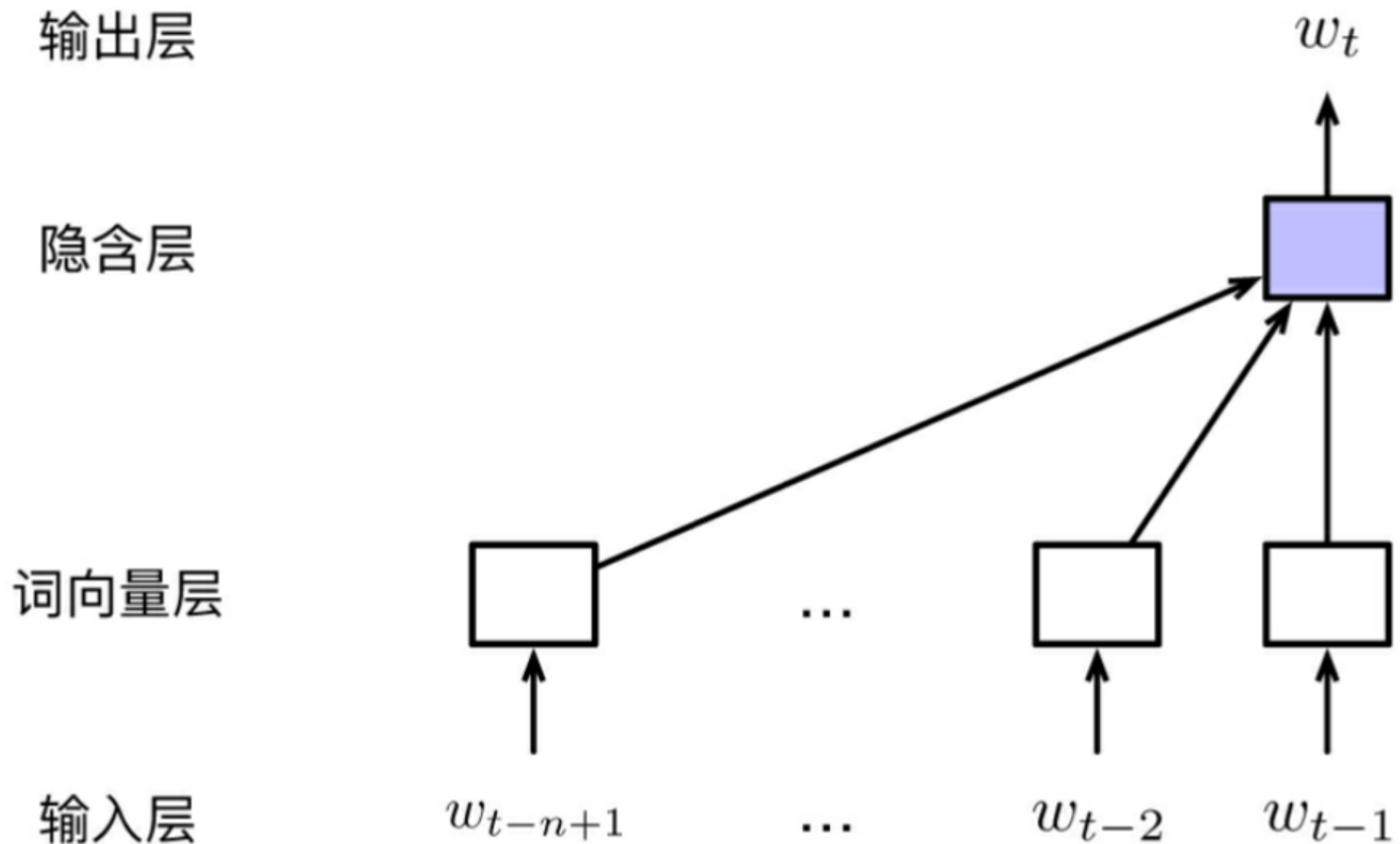
w_{t-n+1}

...

w_{t-2}

w_{t-1}

前馈神经网络语言模型



前馈神经网络语言模型

输出层

隐含层

- 输入层: 当前时刻 t 的历史词序列 $w_{t-n+1:t-1}$ 构成, 主要为离散的符号表示
 - 每个词的独热编码 (One-Hot Encoding)
 - 词在词表中的ID

输入层

w_{t-n+1}

...

w_{t-2}

w_{t-1}

w_t



前馈神经网络语言模型

- 词向量层: 将输入层中的每个词分别映射至一个低维、稠密的实值特征向量
- 词向量层也可以理解为一个查找表(Look-up Table), 获取词向量的过程, 也就是根据词的索引从查找表中找出对应位置的向量的过程。
- $\mathbf{v}_w \in \mathbb{R}^d$:词w的d维词向量 $\mathbf{x} = [\mathbf{v}_{w_{t-n+1}}; \cdots; \mathbf{v}_{w_{t-2}}; \mathbf{v}_{w_{t-1}}]$
- $\mathbf{x} \in \mathbb{R}^{(n-1)d}$:历史序列中所有词向量拼接之后的结果
- $\mathbf{E} \in \mathbb{R}^{d \times |\mathcal{V}|}$:词向量矩阵:

词向量层

输入层



前馈神经网络语言模型

输出层

隐含层:对词向量层 x 进行线性变换与激活

$$h = f(W^{\text{hid}}x + b^{\text{hid}})$$

隐含层

$$x = [v_{w_{t-n+1}}; \dots; v_{w_{t-2}}; v_{w_{t-1}}]$$

词向量层

输入层

w_{t-n+1}

...

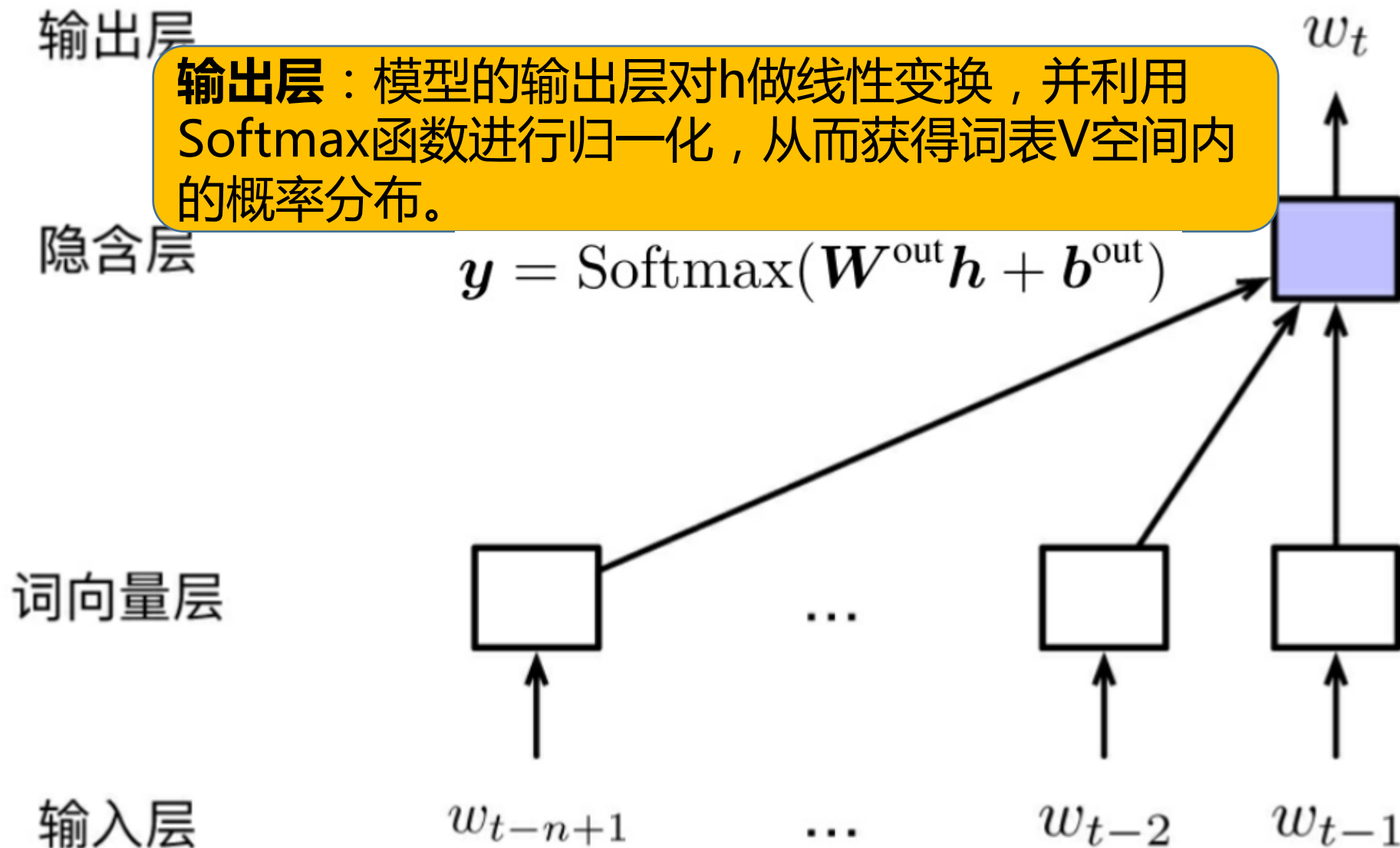
w_{t-2}

w_{t-1}

w_t



前馈神经网络语言模型



前馈神经网络语言模型

前馈神经网络语言模型的自由参数：

□ E ：词向量矩阵

□ W_{hid} 、 b_{hid} ：词向量层与隐含层之间的权值矩阵及偏置项

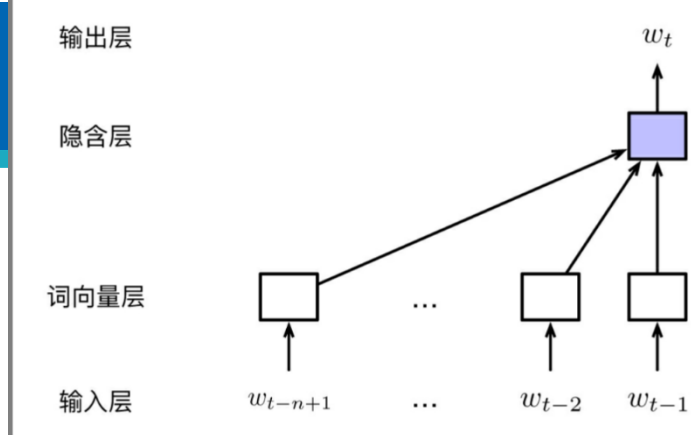
□ W_{out} 、 b_{out} ：隐含层与输出层之间的权值矩阵与偏置项

□ 参数个数： $\theta = \{E, W_{hid}, b_{hid}, W_{out}, b_{out}\}$

$$|\mathbb{V}| \times d + m \times (n - 1)d + m + |\mathbb{V}| \times m + |\mathbb{V}| \\ |\mathbb{V}|(1 + m + d)$$

□ m 和 d 是常数，模型的自由参数数量随词表大小呈线性增长，且 n 的增大并不会显著增加参数的数量

□ 词向量维度 d 、隐含层维度 m 和输入序列长度 $n-1$ 等超参数的调优需要在开发集上进行



模型训练完成后，矩阵 E 则为预训练得到的静态词向量

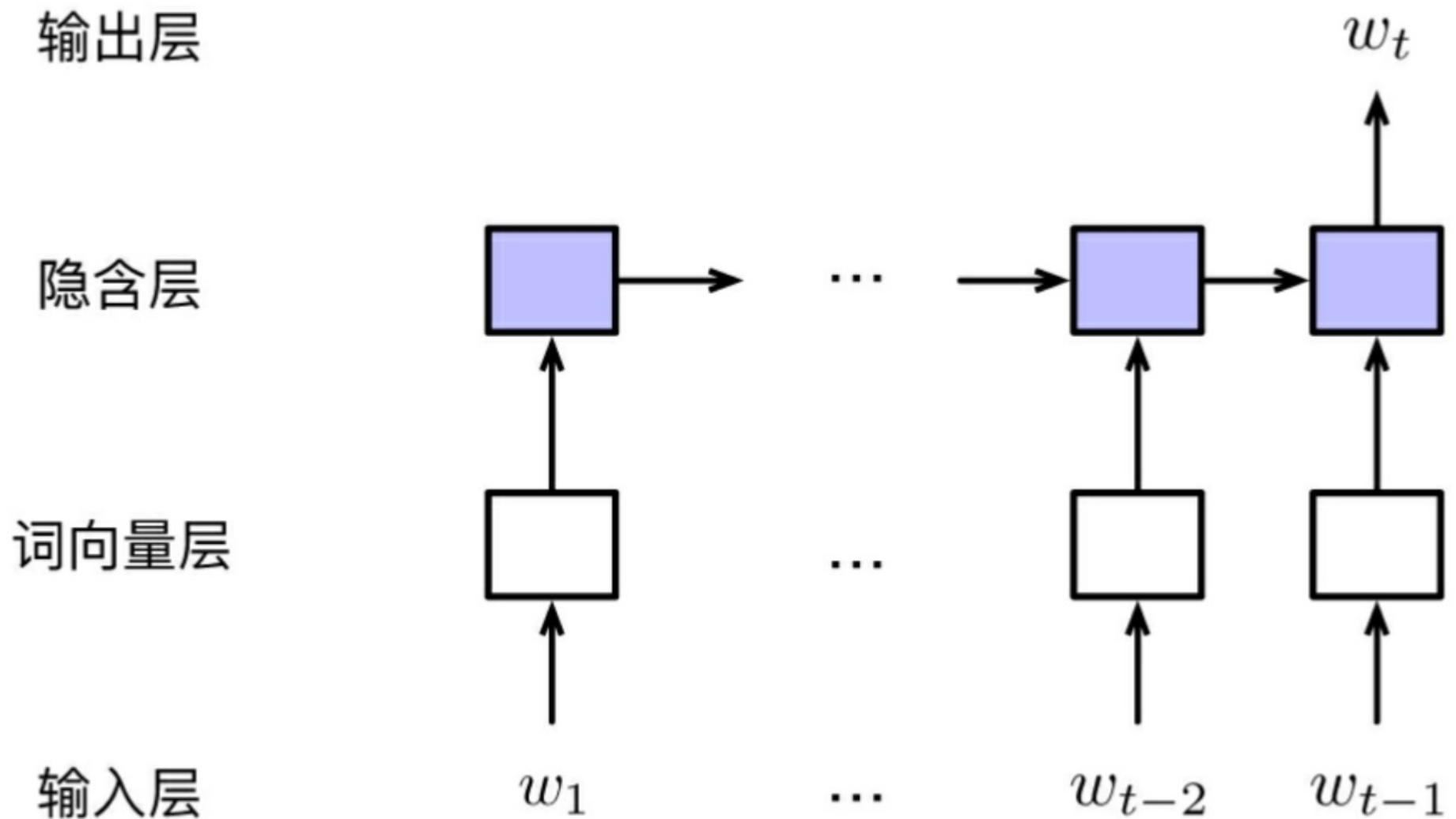
循环神经网络

- ❑ **前馈神经网络语言模型**：一个词的预测需要回看多长的历史是由超参数 n 决定
- ❑ n 是固定的
- ❑ 不同的句子对历史长度 n 的期望往往是变化的
 - ① “他 喜欢 吃 苹果”，根据“吃”容易推测出
 - ② “他感冒了，于是下班之后去了 医院”
- ❑ **循环神经网络语言模型**
 - ❑ 正是为了处理这种不定长依赖而设计的一种语言模型
 - ❑ 用来处理序列数据的一种神经网络
 - ❑ 自然语言正好满足这种序列结构性质

循环神经网络

- 循环神经网络语言模型中的每一时刻都维护一个隐含状态，该状态蕴含了当前词的所有历史信息，且与当前词一起被作为下一时刻的输入。
- 这个随时刻变化而不断更新的隐含状态也被称作记忆(Memory)

循环神经网络



循环神经网络

输出层

隐含层

输入层:与前馈神经网络语言模型不同，由于模型不再受限于历史上下文的长度，所以此时输入层可由完整的历史词序列构成，即

$w_{1:t-1}$

输入层

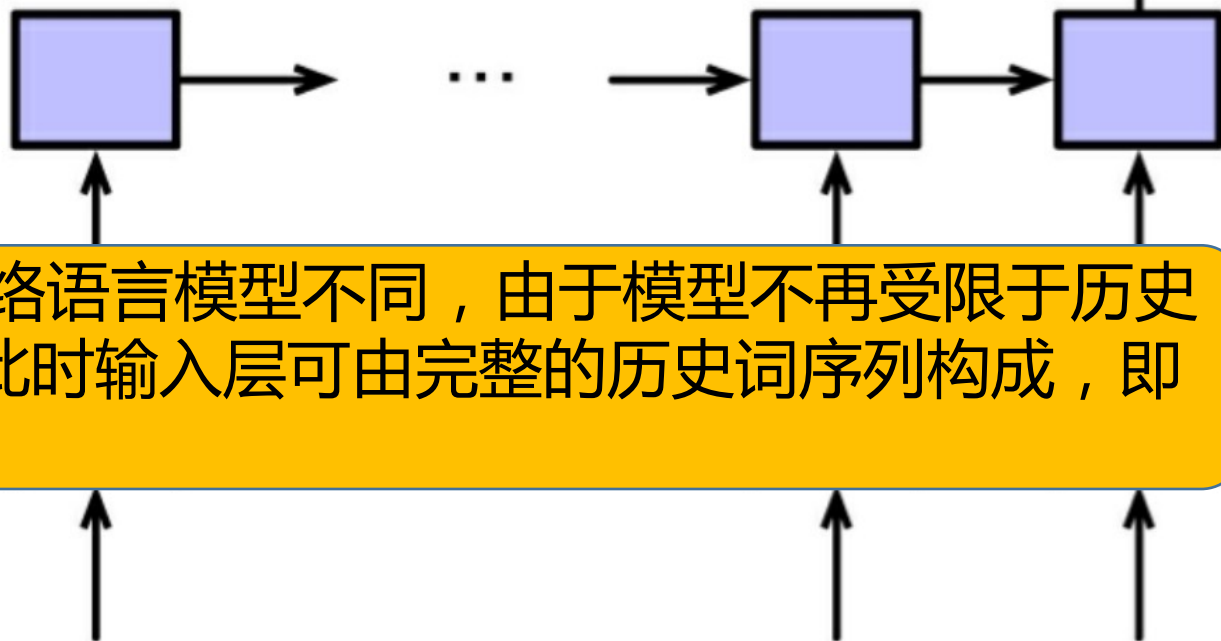
w_1

...

w_{t-2}

w_{t-1}

w_t



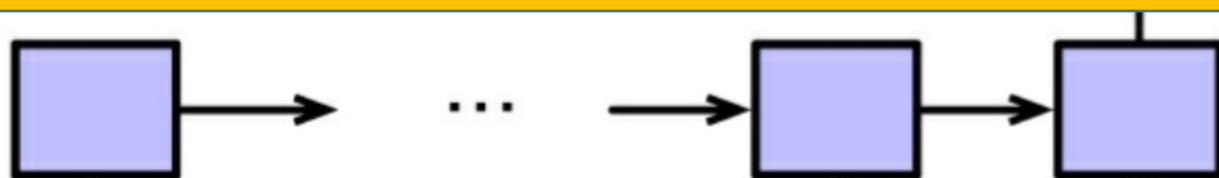
循环神经网络

输出层

u_t

词向量层：输入的词序列映射至相应的词向量表示

隐含层



词向量层



输入层

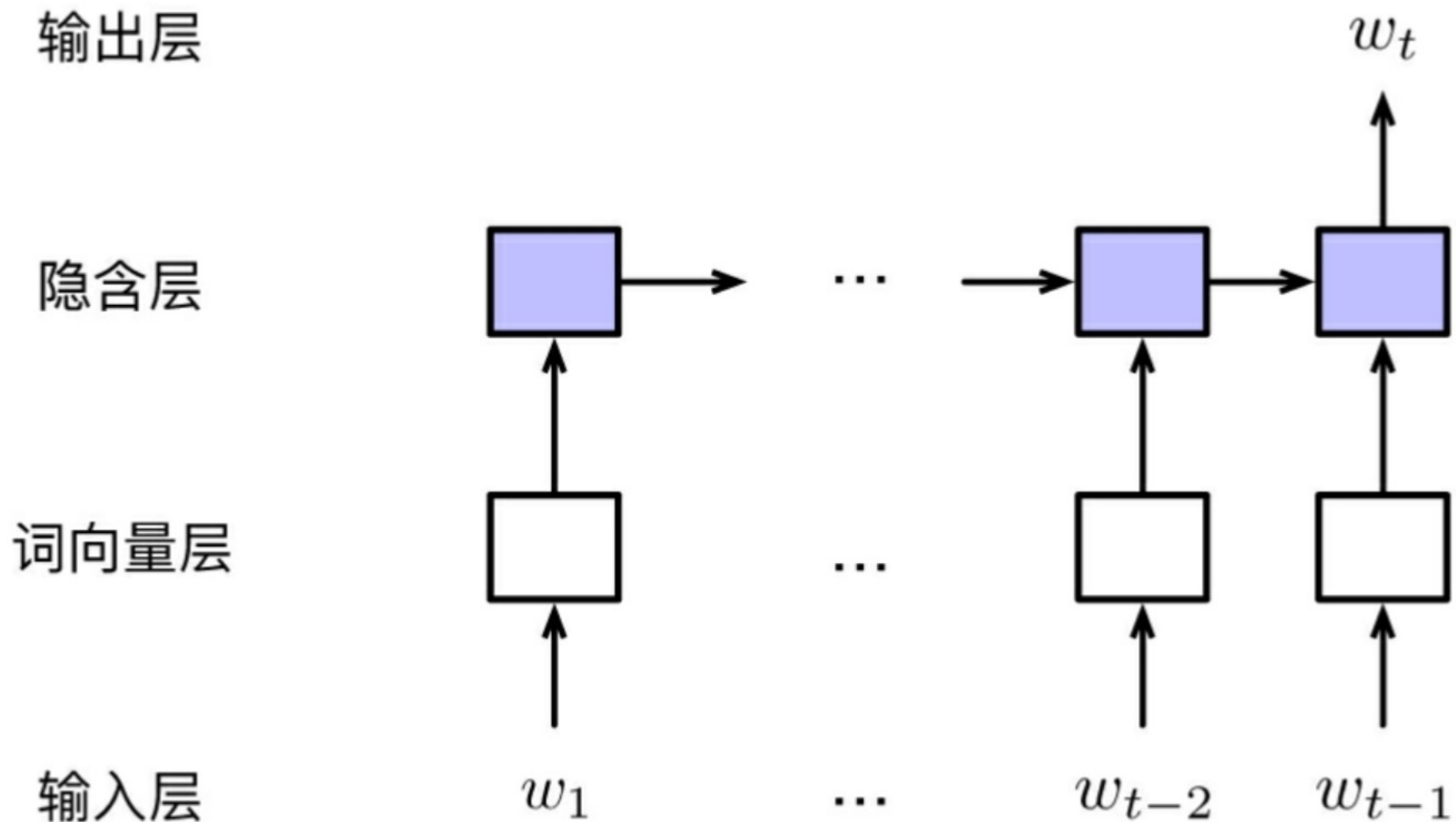
w_1

...

w_{t-2}

w_{t-1}

循环神经网络



循环神经网络

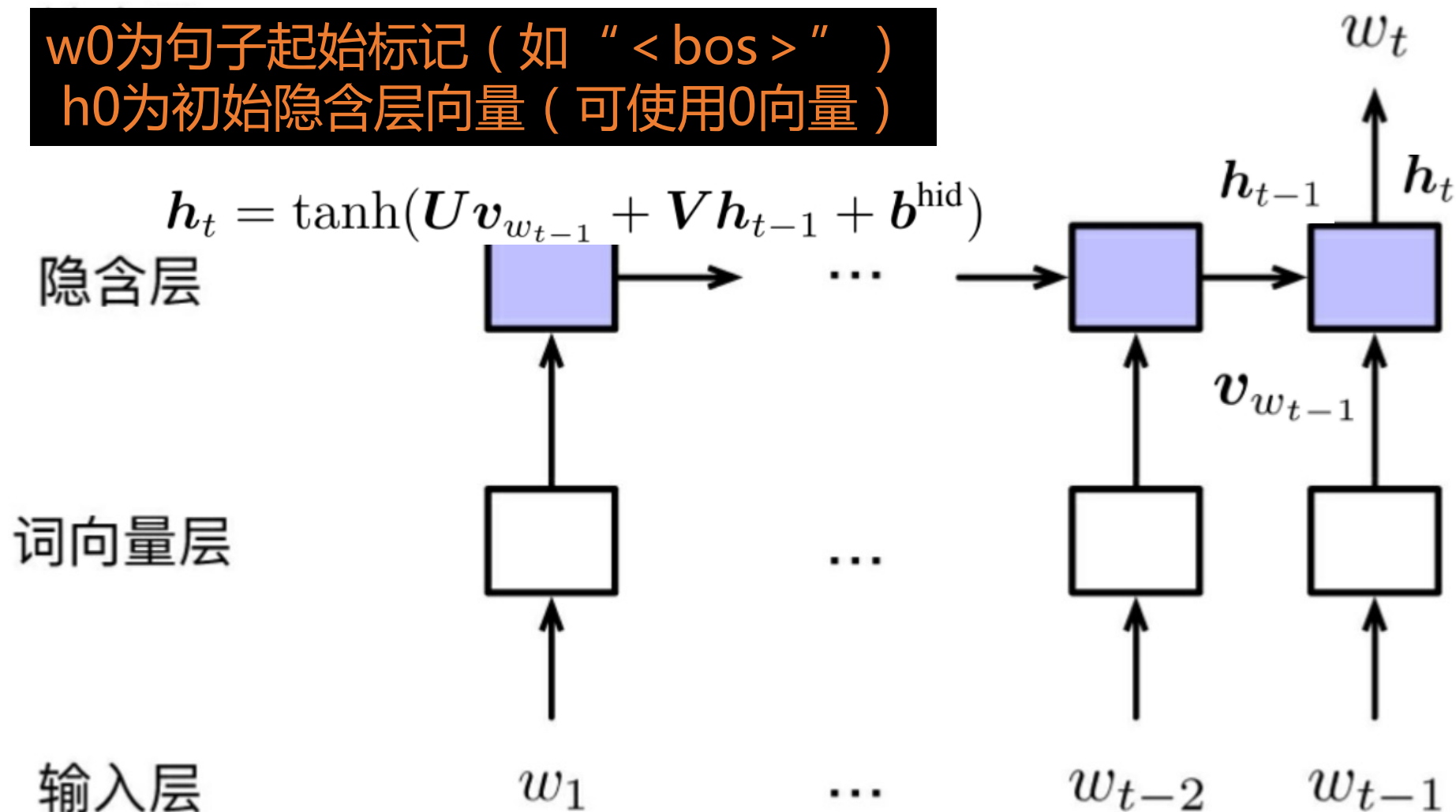
隐含层: 由线性变换与激活函数构成

$$h_t = \tanh(\mathbf{W}^{\text{hid}} \mathbf{x}_t + \mathbf{b}^{\text{hid}})$$

$$\mathbf{x}_t = [\mathbf{v}_{w_{t-1}}; \mathbf{h}_{t-1}]$$

其中 $\mathbf{W}^{\text{hid}} = [\mathbf{U}; \mathbf{V}]$

w_0 为句子起始标记 (如 "< bos >")
 h_0 为初始隐含层向量 (可使用 0 向量)



循环神经网络

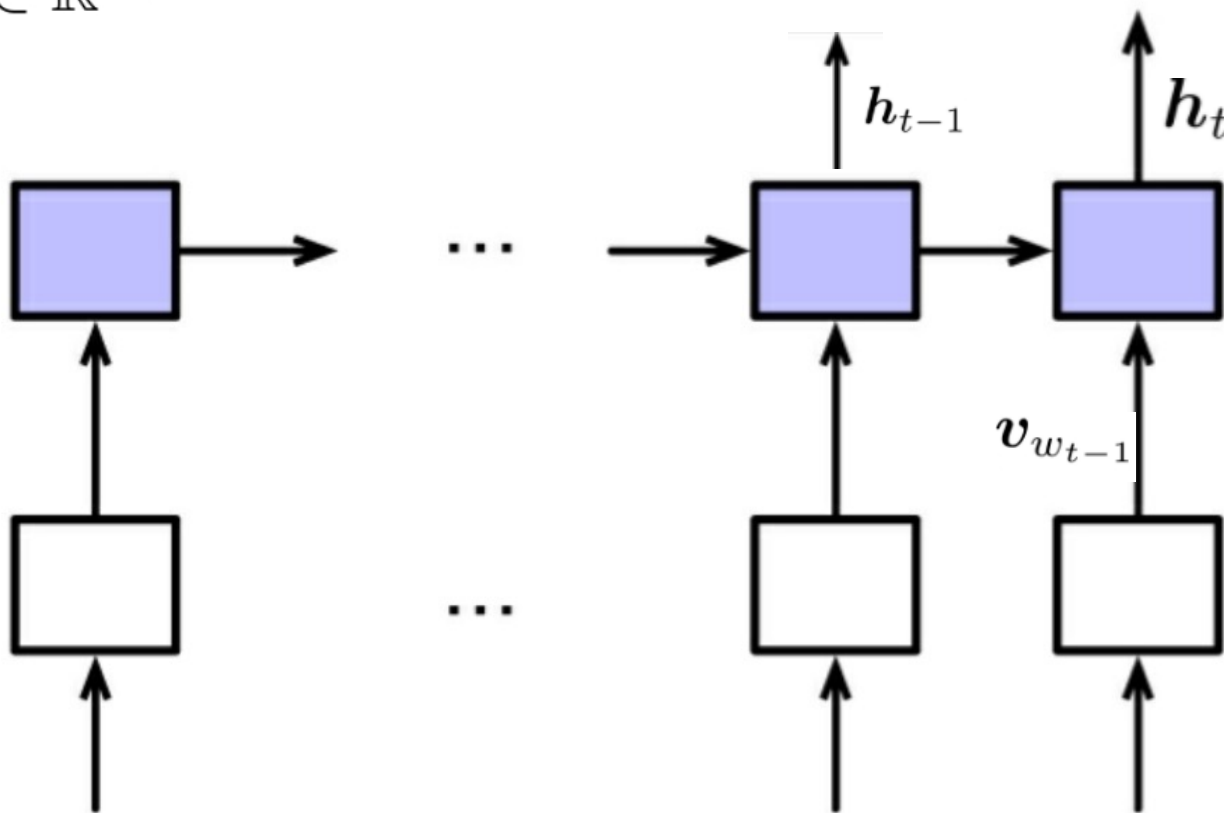
输出层:在输出层计算t时刻词表上的概率分布：

输出层

$$\mathbf{y}_t = \text{Softmax}(\mathbf{W}^{\text{out}} \mathbf{h}_t + \mathbf{b}^{\text{out}})$$

$$\mathbf{W}^{\text{out}} \in \mathbb{R}^{|\mathcal{V}| \times m}$$

隐含层



词向量层

输入层

w_1

$\dots$

w_{t-2}

w_{t-1}

循环神经网络

- ❑ 当序列较长时，训练阶段会存在**梯度弥散**（ Vanishing gradient ）或者**梯度爆炸**（ Exploding gradient ）的风险
- ❑ 为了应对这一问题，以前的做法是在梯度反向传播的过程中按长度进行**截断**（ Truncated Back-propagation Through Time ）从而使得模型能够得到有效的训练，但是与此同时，也减弱了模型对于长距离依赖的建模能力
- ❑ 这种做法一直持续到2015年左右，之后被含有门控机制的循环神经网络如**长短时记忆网络**（ LSTM ）

单词向量的评价方法

如何评价词向量优劣？

- 外部任务评价方法
- 内部任务评价方法
 - 词义相关性
 - 类比推理性的表达能力

单词向量的评价方法

- ❑ 外部任务评价方法：在实际任务中，根据下游任务的性能指标判断
- ❑ 外部任务评价例子：情感分类任务，分类准确率
- ❑ 外部任务评价缺点：
 - ❑ 单词的分布式表示的学习和分类系统的学习有时会分开进行
 - ❑ 要调查单词的分布式表示的如何影响最终的精度
 - ① 需要进行单词的分布式表示的学习(word2vec)
 - ② 再利用这个分布式表示进行另一个机器学习系统的学习(比如进行情感分类的SVM等)
 - ❑ 需要进行两个阶段的学习之后，才能进行评价
 - ❑ 在这种情况下，由于需要调试出对两个系统都最优的超参数，所以非常费时

单词向量的评价方法

如何评价词向量优劣？

- ▣ 外部任务评价方法
- ▣ 内部任务评价方法
 - ▣ 词义相关性
 - ▣ 类比推理性的表达能力

单词向量的评价方法

词义相关性

- 对词义相关性的度量是词向量的重要性质之一
- 根据词向量对词义相关性的表达能力衡量词向量的好坏
- 任意两个词a和b词词义相关性的度量：词向量 w_a 与 w_b ，它们在词向量空间内的余弦相似度

$$\text{sim}(w_a, w_b) = \cos(\mathbf{v}_{w_a}, \mathbf{v}_{w_b}) = \frac{\mathbf{v}_{w_a} \cdot \mathbf{v}_{w_b}}{\|\mathbf{v}_{w_a}\| \|\mathbf{v}_{w_b}\|}$$

- 基于该相关性度量，可完成
 - ① 近义词检索
 - ② 词向量进行定量的评价

单词向量的评价方法

词义相关性：近义词检索

- ❑ 定义以函数实现K近邻查询，利用该函数，可实现在词向量空间内进行
- ❑ 例：GloVe词向量空间内以 “august”“good”为查询词进行近义词检索，可以得到以下结果

```
>>> find_similar_words(pt_embeds, pt_vocab, "august", k=3)
```

```
Query word: august
```

```
cosine similarity=0.8319: september
```

```
cosine similarity=0.8030: july
```

```
cosine similarity=0.7651: june
```

```
>>> find_similar_words(pt_embeds, pt_vocab, "good", k=3)
```

```
Query word: good
```

```
cosine similarity=0.7299: bad
```

```
cosine similarity=0.6923: funny
```

```
cosine similarity=0.6845: tough
```

词义相关性：词向量进行定量的评价

- 利用含有词义相关性的人工标注作为黄金标准，对词向量进行定量评价
- 常用的评价数据集——WordSim353为例
 - 该数据集包含353个英文词对，每个词对由16位标注者给出[0, 10]区间内的一个数值，最后取平均值作为该词对的词义相似度

表5-1 WordSim353数据集中的词义相似度标注示例

单词 1	单词 2	相似度
love	sex	6.77
stock	jaguar	0.92
money	cash	9.15
development	issue	3.97
lad	brother	4.46

- 由词向量计算得到的相似度值与人工标注值之间的相关系数(如Pearson相关系数)即可作为词向量评价的标准

■词的类比性: 对词向量的另一种常用的内部任务评价方法

□对词向量在向量空间内的分布进行分析可以发现，对于语法或者语义关系相同的两个词对 (w_a, w_b) 与 (w_c, w_d) ，它们的词向量在一定程度上满足： $v_{w_b} - v_{w_a} \approx v_{w_d} - v_{w_c}$

□语义和语法例子：

$$v_{\text{WOMAN}} - v_{\text{MAN}} \approx v_{\text{QUEEN}} - v_{\text{KING}}$$

(5-23)

$$v_{\text{QUEENS}} - v_{\text{QUEEN}} \approx v_{\text{KINGS}} - v_{\text{KING}}$$

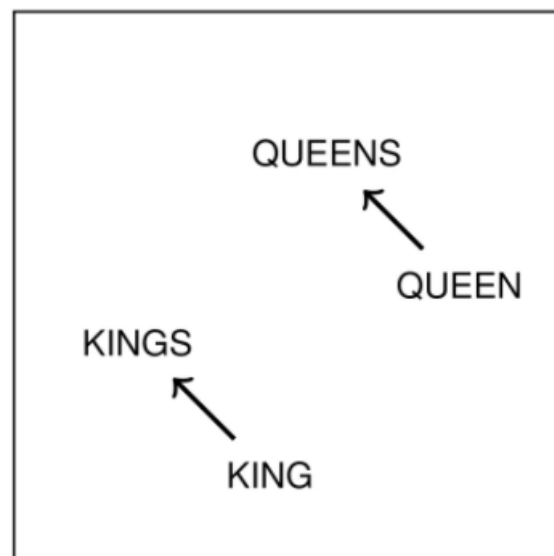
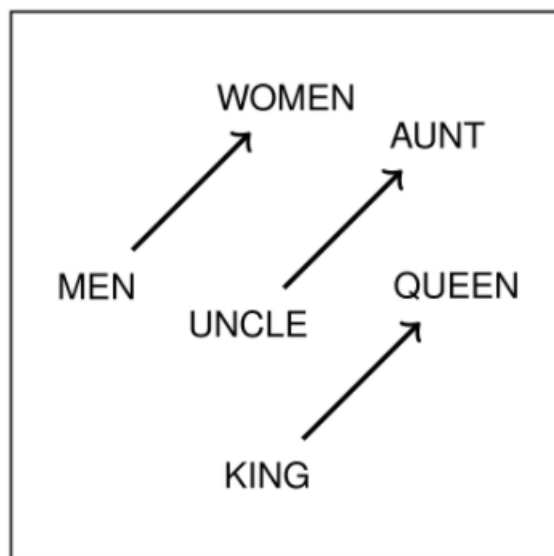


图5-6 词向量空间内的语义和语法类比推理性质示例

词向量的评价方法

- 利用 $\mathbf{v}_{w_b} - \mathbf{v}_{w_a} \approx \mathbf{v}_{w_d} - \mathbf{v}_{w_c}$ ，可进行词与词之间的关系推理，从而回答诸如“ w_a 之于 w_b ，相当于 w_c 之于？”的问题

$$w_d = \arg \min_w (\cos(\mathbf{v}_w, \mathbf{v}_{w_c} + \mathbf{v}_{w_b} - \mathbf{v}_{w_a}))$$

- 利用knn函数，即可实现
- 一般来说，词向量在以上评价方法中的表现与训练数据的来源及规模、词向量的维度等因素密切相关。在实际应用中，需要根据词向量在具体任务中的表现来选择

单词向量的评价方法

- 以word2vec的模型、单词的分布式表示的维数和语料库的大小为参数进行了比较实验，结果如下：

模型	维数	语料库的大小	Semantics	Syntax	Total
CBOW	300	16 亿	16.1	52.6	36.1
skip-gram	300	10 亿	61	61	61
CBOW	300	60 亿	63.6	67.4	65.7
skip-gram	300	60 亿	73.0	66.0	69.1
CBOW	1000	60 亿	57.3	68.9	63.7
skip-gram	1000	60 亿	66.1	65.1	65.6

- Semantics列**：显示的是推断单词含义的类推问题(像 “king: queen=actor: actress”这样询问单词含义的问题)的正确率
- Syntax列**：询问单词形态信息的问题，比如 “bad:worst=good:best”

单词向量的评价方法

模型	维数	语料库的大小	Semantics	Syntax	Total
CBOW	300	16 亿	16.1	52.6	36.1
skip-gram	300	10 亿	61	61	61
CBOW	300	60 亿	63.6	67.4	65.7
skip-gram	300	60 亿	73.0	66.0	69.1
CBOW	1000	60 亿	57.3	68.9	63.7
skip-gram	1000	60 亿	66.1	65.1	65.6

- ❑ 模型不同，精度不同(根据语料库选择最佳的模型)
- ❑ 语料库越大，结果越好(始终需要大数据)
- ❑ 单词向量的维数必须适中（太大会导致精度变差）

单词向量的评价方法

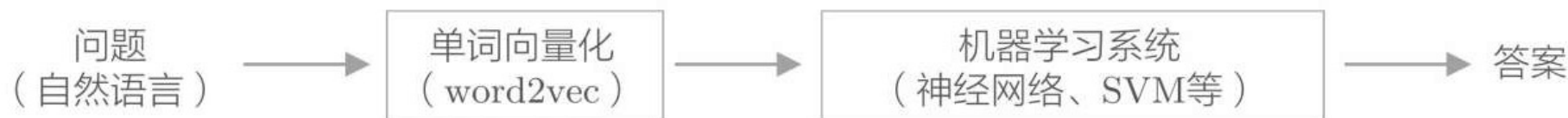
- 基于类推问题可以在一定程度上衡量 “是否正确理解了单词含义或语法问题”
- 在NLP的应用中，能够高精度地解决类推问题的单词的分布式表示应该可以获得好的结果
- 单词的分布式表示的优劣对目标应用贡献多少(或者有无贡献)，取决于待处理问题的具体情况
 - 比如应用的类型或语料库的内容等
 - 类推问题的评价高，不能保证目标应用的结果就一定好!!!!

词向量的应用

- ❑ 在解决NLP任务时，一般不会使用word2vec从零开始学习单词的分布式表示，而是
 - ① 先在大规模语料库（Wikipedia、Google News等文本数据）上学习
 - ② 然后将学习好的分布式表示应用于某个单独的任务
- ❑ 比如，在文本分类、文本聚类、词性标注和情感分析等自然语言处理任务中，第一步的单词向量化工作就可以使用学习好的单词的分布式表示。

词向量的应用

- 单词的分布式表示的优点：可以将单词转化为**固定长度**的向量
- **文档**(单词序列)转化为**固定长度**的向量(**bag-of-words**)方法：
 - **步骤**：
 - ① 把文档的各个单词转化为分布式表示
 - ② 求单词分布式表示的总和
 - **缺点**：未考虑单词顺序的模型
- 将单词和文档转化为固定长度的向量非常重要：将自然语言转化为向量后，就可以使用常规的机器学习方法（神经网络、SVM等）



词向量应用

预训练词向量应用：

- ① 作为词的特征表示直接用于下游任务
 - ② 作为模型参数在下游任务的训练过程中进行精调(Fine-tuning)
- ▣ 在通常情况下，两种方式都能够有效地提升模型的泛化能力

词向量的应用

邮件自动分类系统



- ① 需要从**收集数据(邮件)** 开始
- ② 收集用户发送的邮件，并人工对邮件**进行标注**，打上表示3类情感的标签(positive/neutral/negative)
- ③ 标注工作结束后，**用学习好的word2vec**将邮件转化为向量
- ④ 将**向量化的邮件**及其**情感标签**输入某个情感分类系统(SVM或神经网络等) 进行学习

词向量的应用

- 多层感知器、循环神经网络等,完成情感分析以及词性标注等自然语言处理任务,均使用了随机初始化的词向量层实现由离散词表示到连续向量表示的转换。
- 利用已预训练好的词向量的替换随机初始化的词向量

MLP模型使用预训练词向量进行情感分类

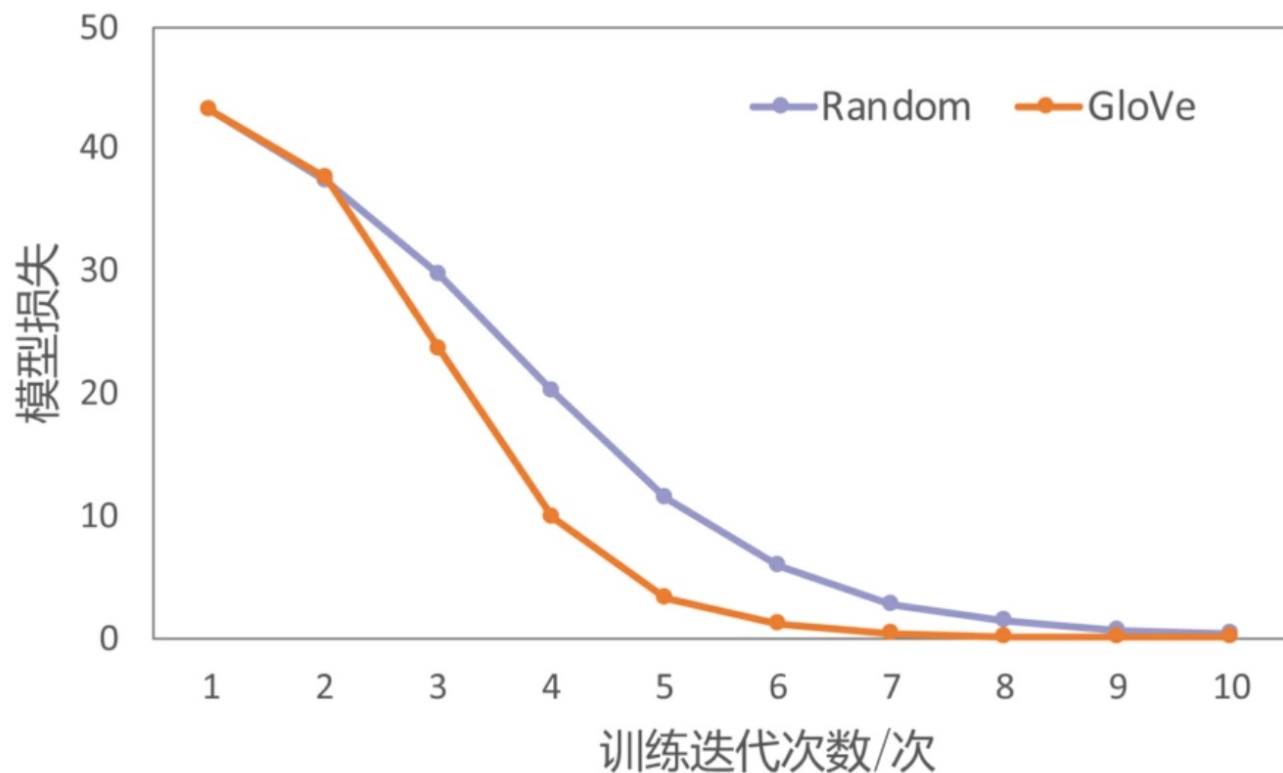
```
class MLP(nn.Module):
```

```
    def __init__(self, vocab, pt_vocab, pt_embeddings, hidden_dim, num_class):
        super(MLP, self).__init__()
        # 与预训练词向量维度保持一致
        embedding_dim = pt_embeddings.shape[1]
        # 词向量层
        vocab_size = len(vocab)
        self.embeddings = nn.EmbeddingBag(vocab_size, embedding_dim)
        self.embeddings.weight.data.uniform_(-0.1, 0.1)
        # 使用预训练词向量对词向量层进行初始化
        for idx, token in enumerate(vocab.idx_to_token):
            pt_idx = pt_vocab[token]
            # 只初始化预训练词典中存在的词
            # 对于未出现在预训练词典中的词，保留其随机初始化向量
            if pt_idx != pt_vocab.unk:
                self.embeddings.weight[idx].data.copy_(pt_embeddings[pt_idx])

        # 线性变换：词向量层->隐含层
        self.fc1 = nn.Linear(embedding_dim, hidden_dim)
        # 线性变换：隐含层->输出层
        self.fc2 = nn.Linear(hidden_dim, num_class)
        # 使用ReLU激活函数
        self.activate = F.relu
```

MLP模型使用预训练词向量进行情感分类：实验结果

- 数据集：NLTK sentence_polarity, 使用正负各1,000个样本
- 下图展示了其与使用随机初始化词向量层的模型在训练过程中损失函数的变化曲线



- 预训练词向量能够显著加快模型训练时的收敛速度
- 10轮迭代之后，模型在测试集上的准确率为70%，相比于使用随机初始化词向量层的模型（67%），取得较为显著的提升

Thanks!
Q&A ?

wuxianyan@njau.edu.cn